

Разработка и анализ прототипа сервиса MediConnect с помощью LLM-средств вайб-кодинга

Лабораторная работа · Кошкарёв Кирилл Павлович

1. Цель и задачи работы

Цель: разработать функциональный прототип сервиса *MediConnect* — универсальной платформы обмена медицинскими данными между пациентами, врачами и медицинскими учреждениями — с помощью средств вайб-кодинга, основанных на больших языковых моделях, и проанализировать полученный результат на предмет качества, производительности и пригодности к развёртыванию.

Задачи:

1. Сформулировать сценарии использования сервиса и его первичную архитектуру с использованием LLM (на основе ранее построенных UML-диаграмм компонентов, активностей и последовательностей).
2. Спроектировать структуру данных предметной области (пациенты, врачи, медицинские записи, согласия, уведомления, аудит).
3. С помощью LLM-инструмента вайб-кодинга реализовать прототип, реализующий ключевые сценарии: вход с разделением ролей, управление медкартой, выдача и отзыв согласий, передача данных врачу с контролем доступа, журнал аудита.
4. Провести функциональное тестирование и анализ полученного сервиса: выявить дефекты, оценить производительность и качество кода, рассмотреть варианты развёртывания.
5. Сформулировать рефлексивные выводы по результатам работы.

2. Ссылки на материалы

Чат с большой языковой моделью

[вставьте сюда ссылку на чат — например: <https://claude.ai/share/<...>> или <https://chatgpt.com/share/<...>>]

Репозиторий проекта (обязательно)

[ссылка на ветку репозитория с исходным кодом прототипа, например: <https://github.com/<user>/PRIS-MediConnect/tree/lab-vibecoding>]

Проект в среде вайб-кодинга (по возможности)

[[Replit/Cursor/StackBlitz/CodeSandbox](#) — ссылка, если есть]

Развёрнутый работающий сервис либо видео-демо

[ссылка на размещение, например, [GitHub Pages](#) / [Netlify](#) / [Vercel](#); либо ссылка на видеофайл с демонстрацией работы]

3. Сценарии и архитектура

В качестве основы взята архитектура из ранее выполненных работ (диаграмма компонентов, диаграмма активностей и диаграмма последовательности): мобильный клиент с локальным кэшем взаимодействует с серверной платформой через API Gateway, который маршрутизирует запросы к сервисам Auth, Medical Records, Consent, Notification и Audit. Прототип воспроизводит эту структуру в рамках одностраничного веб-приложения, эмулируя серверные сервисы в JavaScript-модуле и используя `localStorage` в качестве БД и оффлайн-кэша.

Ключевые сценарии, реализованные в прототипе:

- **Вход с проверкой роли:** пользователь явно указывает роль (пациент / врач / администратор) и не может войти с несоответствующей роли учётной записью — попытка фиксируется в журнале аудита (`login_role_mismatch`).
- **Просмотр медкарты пациентом:** личные данные, аллергии, хронические заболевания, список анализов и приёмов.
- **Управление согласиями:** пациент в один клик выдаёт или отзывает согласие конкретному врачу.
- **Передача данных врачу:** ключевое бизнес-ограничение — без активного согласия отправка блокируется и логируется (`share_blocked_no_consent`); при наличии согласия создаётся уведомление врачу.
- **Работа врача:** список пациентов, выдавших согласие, просмотр уведомлений и создание записей о приёмах/анализах/назначениях.
- **Администрирование:** журнал аудита с фильтром, реестр пользователей, статус сервисов системы.

Структура данных

Сущность	Поля	Назначение
User	id, login, pwd, role, name, dob, allergies, conditions, specialty	Учётная запись пациента/врача/админа
Record	id, patientId, type, date, text	Медицинская запись (анализ / приём / назначение)
Consent	id, patientId, doctorId, granted, ts	Согласие пациента на доступ конкретного врача
Notification	id, doctorId, patientId, recordId, ts, seen	Уведомление врача о новых данных
AuditEvent	id, ts, actorId, action, target, meta	Запись журнала безопасности

4. Скриншоты прототипа

MediConnect

Вход в MediConnect

Платформа обмена медицинскими данными между пациентами, врачами и клиниками.

Пациент

Врач

Администратор

Логин

Пароль

Войти

► Демо-аккаунты

Рис. 1. Экран входа с выбором роли (Auth Service)

MediConnect

Иванов И. И. · Пациент

Выйти

Моя медкарта

Личные данные, аллергии, хронические заболевания.

ФИО

Иванов И. И.

Дата рожд.

1986-03-12

Аллергии

Пенициллин

Хронич.

Гипертония

Анализы и записи

Результаты анализов и приёмов из локального кэша / API.

Анализ

Общий анализ крови — норма (Hb 145, Лейк 6.2)

2026-04-12

Приём

Приём терапевта: жалобы на давление, рекомендован холтер

2026-04-15

Согласия на доступ

Управляйте, кто из врачей имеет доступ к вашим данным. Без согласия передача запрещена.

Смирнов А. В.

Терапевт · обновлено 18.04.2025, 21:13:20

Доступ есть

Отозвать

Кузнецова Е. П.

Кардиолог

Нет доступа

Дать согласие

Передать данные врачу

Сценарий из диаграммы последовательности: проверка согласия → передача → уведомление.

Врач

Что передать

Смирнов А. В. — Терапевт

Анализ · 2026-04-12 · Общий анализ крови — норма (Hb 145, Лейк...

Отправить

Рис. 2. Кабинет пациента: медкарта, анализы, управление согласиями, передача данных

MediConnect

Смирнов А. В. · Врач

Выйти

Мои пациенты

Список пациентов, которые предоставили вам согласие на доступ.

Иванов И. И.

Дата рожд.: 1986-03-12 · Аллергии: Пенициллин · Записей: 2

Согласие активно

Уведомления

Новые данные, переданные пациентами.

Иванов И. И.

Приём: Приём терапевта: жалобы на давление, рекомендован холтер

30.04.2025, 11:00:00

Новое

Добавить запись о приёме

Пациент

Иванов И. И.

Тип

Приём

Описание

Жалобы, диагноз, рекомендации...

Сохранить

Рис. 3. Кабинет врача: пациенты с согласием, уведомления, добавление записи

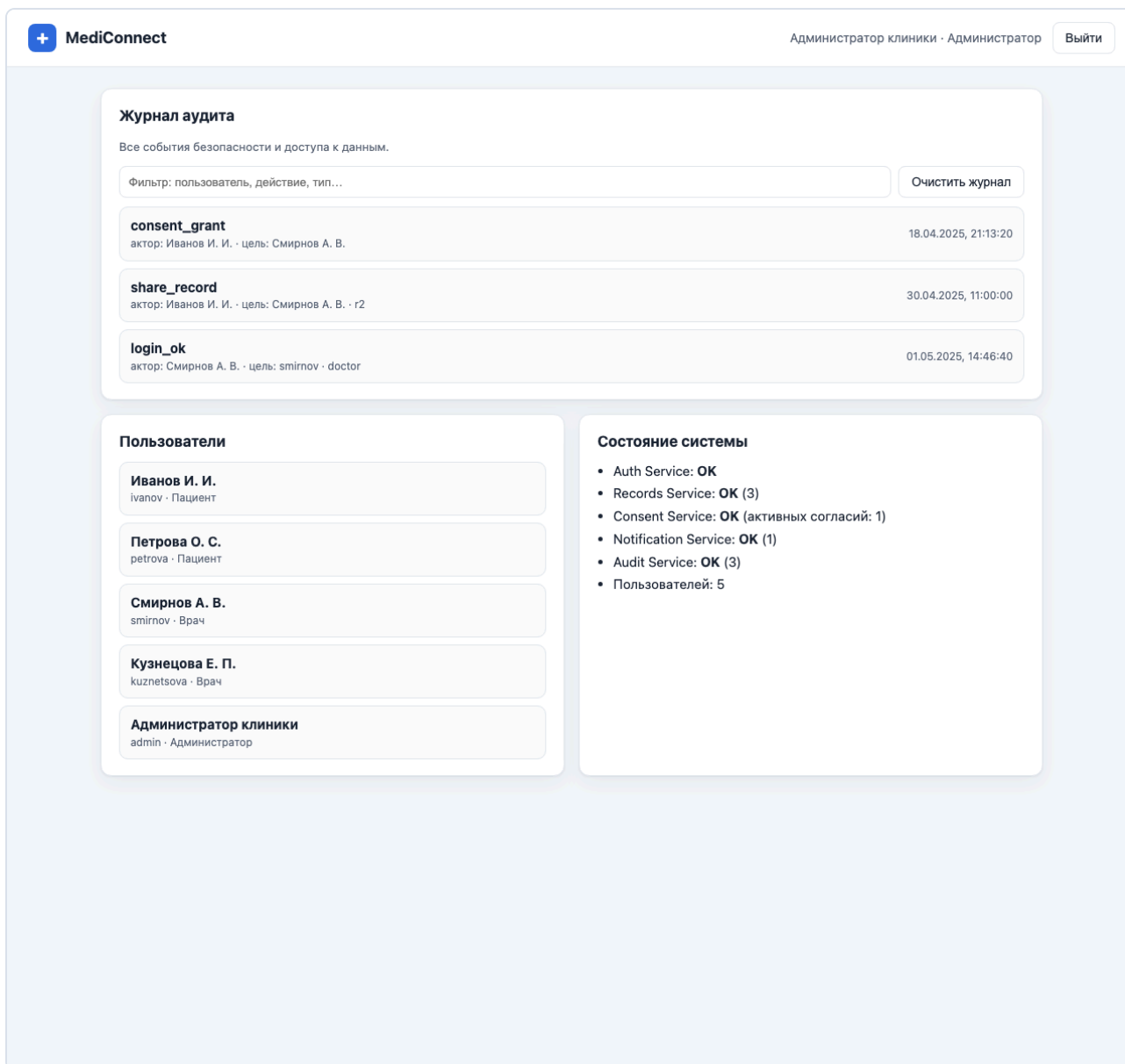


Рис. 4. Кабинет администратора: журнал аудита, пользователи, состояние сервисов

5. Анализ результата

Степень успеха разработки. Прототип покрывает все ключевые сценарии из ранее построенных диаграмм: проверку токена/роли при входе, чтение медкарты с локальным кэшем, обязательную проверку согласия перед передачей данных и формирование уведомления врачу. Тестирование вручную в трёх ролях (пациент, врач, администратор) показало, что позитивные сценарии выполняются корректно. Из выявленных при тестировании особенностей: попытка передачи данных без согласия корректно блокируется и фиксируется в аудите; вход с правильным паролем, но «чужой» ролью также отклоняется; согласие можно выдать и отозвать, и оба события попадают в журнал. Видимых дефектов в основных пользовательских сценариях не обнаружено.

Производительность и эффективность. Общий объём исходного кода — 787 строк (HTML 160, CSS 211, JS 416) и около 38 КБ без сжатия. На локальном статическом сервере время отклика на запросы `index.html` и `app.js` стабильно укладывается в 1

мс, общий вес страницы ниже 25 КБ — приложение работает мгновенно даже на слабых устройствах. Поскольку всё состояние хранится в `localStorage`, нагрузка на память минимальна (десятки КБ), а оффлайн-сценарий работает «из коробки», что соответствует требованию к мобильному клиенту с локальным кэшем.

Качество кода. Кодовая база компактна и однородна: один JS-файл реализует роутер, эмуляцию пяти сервисов и три экрана; HTML-шаблоны вынесены в `<template>`; стили изолированы в CSS с использованием CSS-переменных. Применено экранирование текста при подстановке в DOM, что снижает риск XSS. Слабые стороны — нет автоматизированных тестов, отсутствует разделение модулей, пароли хранятся в открытом виде (что приемлемо для прототипа, но требует вынесения в реальный Auth-сервис при продакшене).

Возможности развёртывания. Прототип представляет собой статический сайт без бэкенда, поэтому разворачивается на любой бесплатный хостинг — GitHub Pages, Netlify, Vercel, Cloudflare Pages — буквально за один шаг. Для перехода к промышленному варианту достаточно вынести JS-сервисы в реальные микросервисы за API Gateway и заменить `localStorage` на защищённую БД с шифрованием PHI согласно требованиям 152-ФЗ и FHIR-совместимости.

6. Рефлексивное заключение

В ходе работы удалось пройти полный цикл: от концептуальных UML-диаграмм, выполненных в предыдущих лабораторных, до работающего прототипа, реализующего бизнес-логику доменной области. Главное, что получилось — связать архитектурные артефакты с конкретным кодом: сервисы из диаграммы компонентов превратились в JS-модули, а сценарий «проверка согласия → передача → уведомление» из sequence-диаграммы напрямую отражён в обработчике формы передачи данных.

Основные сложности были связаны не столько с генерацией кода, сколько с формулированием контекста для LLM: модель хорошо справляется с типовыми UI-блоками, но требует чёткой постановки бизнес-правил (прежде всего проверки согласия и журналирования событий безопасности), иначе склонна упрощать предметную область. Кроме того, сгенерированный код приходилось проверять на корректность экранирования и непротиворечивость состояний — это та часть, которую LLM выполняет менее надёжно, чем рутинную верстку.

Из новой пользы — стало явным, что вайб-кодинг даёт ощутимый выигрыш именно тогда, когда есть проработанная архитектура: при наличии диаграмм компонентов и последовательностей переход к коду занимает часы, а не дни, и итоговый прототип сохраняет связь с моделью. Без архитектуры тот же инструмент склонен генерировать «средний по больнице» CRUD без учёта медицинской специфики (согласия, аудита, оффлайн-режима).